A. AKSHAYA
CSD
24NU1A4404

DATA ENGINEERING ASSIGNMENT - 05

Q1. Write in detail the life of a query? Discuss in detail the three types of windows used in streaming queries and processing with examples.

A) LIFE OF A QUERY

The life of a query refers to the complete journey from when a user submits a query to when the result is produced. It involves multiple stages;

* Query Submission - The user writes & submits a query in SQL or another query language.

Eg: Retrieving records with

* Parsing and Validation - The system checks syntax & semantics. It ensures that tables and columns exist & the query is correctly written.

Eg: Valid query executes successfully

* Logical Plan Creation - The query is converted into an abstract representation (logical plan), including operations like filtering, projection, and aggregation.

Eg: Filter → Select → Scan

* Query Optimization - The optimizer improves efficiency by rearranging operations & minimizing resource usage.

Eg: Applying filters early

* Physical Plan Generation - The logical plan is converted into a physical execution strategy.

Eg: choosing hash join vs nested loop join.

* Execution - The query runs on the execution engine.

Eg: Batch processing for historical data.

* Result Delivery - The final result is returned or stored for further use.

Eg: Display in dashboards.

WINDOWS IN STREAMING QUERIES

Streaming data is continuous, so it is processed using windows.

* Tumbling Window -: A fixed-size, non-overlapping window where each record belongs to only one window.

Eg: • Sales calculated every 5 minutes
• Hourly website traffic analysis

Use Case: Periodic reports and fixed interval metrics

* Sliding Window -: A fixed-size window that overlaps with others and slides at a defined interval.

Eg: • Moving average of temperature over last 10 minutes
• Traffic analysis upd

Use Case: Trend analysis & real-time monitoring.

* Session Window - A dynamic window based on user activity, separated by inactivity gaps.

Eg : • User session ends after 30 minutes of inactivity
• Grouping clicks into browsing sessions

Use case : User behavior tracking & session analysis.

Q2. Briefly discuss the pros and cons of the following update patterns : (a) truncate & reload (b) insert only. Differentiate between hard delete & soft delete.

A) (a) Truncate and Reload -

This method deletes all existing data & reloads it from the source.

• Pros :
→ Simple to implement
→ Ensures clean & consistent data.

• Cons :
→ Inefficient for large datasets.
→ Causes downtime
→ No history maintained

Examples :

• Daily refresh of sales table
• Rebuilding a dimension table

(b) Insert Only -

In this pattern, only new data is appended without modifying existing records.

Pros :

- High performance & scalability
- Maintains historical data

Cons :

- Data duplication possible
- Difficult to correct errors.

Examples :

- Logging systems
- Event streaming pipelines

* Hard Delete vs Soft Delete :

→ Hard Delete -

Data is permanently removed from the database.

Examples :

- Removing user records permanently
- Deleting obsolete entries.

→ Soft Delete -

Data is marked as deleted but not physically removed.

Examples :

- Using a flag like is_deleted = true
- Maintaining audit logs.

Q3. (a) What is normalization? Discuss 1NF, 2NF with examples and illustrate the purpose of de-normalization with examples in analytics.

(b) Define reverse ETL and discuss its role in modern data architectures.

A) (a) NORMALIZATION -

Process of organizing data to :

- Reduce redundancy
- Improve consistency

FIRST NORMAL FORM (1NF) -

- No repeating groups
- Atomic values

Eg :-

ID	Phones
1	123, 456

$\xrightarrow{1NF}$

ID	Phone
1	123
1	456

SECOND NORMAL FORM (2NF) -

- Must be in 1NF
- No partial dependency.

Eg :-

OrderID ProductID ProductName

Here,

ProductName depends only on ProductID → violation

Fix :

→ split into tables

PURPOSE OF DENORMALIZATION

Combining tables to improve read performance.

- Analytics queries need fast reads
- Joins are expensive

Eg:

Customer + Orders joined into one table

(b) Reverse ETL

Reverse ETL moves data from data warehouse → operational systems.

Traditional Flow:

Source → ETL → Warehouse

Reverse ETL:

Warehouse → CRM / Apps

Role in Modern Data Architecture:

- Operational Analytics -
  → Use warehouse data in real-time apps
- Customer Personalization -
  → Send user behavior data to marketing tools